

Biomed Data School: Terminal for Bioinformatics

A practical Unix and HPC reference for biomedical scientists: from folders and files to FASTQ, SRA, QC, alignments, variants, RNA-seq, genome assembly, logs, environments and reproducible workflows.

0. Mental model

A terminal command usually has this shape:

command [options] input_file > output_file

Examples:

```
grep 'BRCA1' genes.tsv > brca1_matches.tsv
```

```
fastqc raw_data/*.fastq.gz -o results/fastqc
```

Symbol	Meaning
.	Current directory
..	Parent directory
~	Your home directory
*	Wildcard: match many filenames
	Pipe: send output of one command into another
>	Write output to a new file, replacing existing content
>>	Append output to an existing file
2>	Redirect error messages
Ctrl+C	Stop a running command
Tab	Autocomplete a command or filename

1. Work only inside a clean project folder

```
mkdir -p
project/{raw_data,data,metadata,references,results,scripts,logs,notebooks}
cd project
pwd
ls -lh
```

Folder	Purpose
raw_data/	Original downloaded data. Treat as read-only. Never manually edit.
data/	Cleaned or derived data produced from raw inputs.
metadata/	Sample sheets, phenotype tables, contrasts, batch information.
references/	Genomes, transcriptomes, annotations, indexes and checksums.
results/	Tool outputs: QC, counts, alignments, plots and reports.
scripts/	Reusable shell scripts and small pipeline wrappers.
logs/	Standard output, errors, versions and command provenance.

Rule: never analyse serious biomedical data in Downloads, Desktop or a random cluster directory. A clean folder structure is already part of reproducibility.

2. Navigation, file operations and safe deletion

Command / pattern	Use
pwd	Print current directory.
ls -lh	List files with human-readable sizes.
ls -lah	Also show hidden files such as .bashrc or .gitignore.
cd folder/ ; cd .. ; cd -	Move into folder, up one level or back to the previous folder.
mkdir -p results/fastqc	Create nested folders without error if they already exist.
cp file backup.txt ; cp -r dir backup/	Copy files and folders.
mv old new	Rename or move.
rm -i file	Delete with confirmation. There is no recycle bin.
rm -r folder/	Delete a directory recursively. Dangerous.
touch notes.txt	Create an empty file or update timestamp.
nano script.sh ; vim script.sh	Edit text files in terminal.
chmod +x script.sh	Make a script executable.

```
# Safe deletion pattern
ls *.tmp # first inspect exactly what would match
rm -i *.tmp # then delete interactively, if correct
```

```
# Avoid spaces in filenames. If you have them, quote paths:
less "sample 1.fastq.gz"
```

3. Inspect files before analysis

Command / pattern	Use
file sample.fastq.gz	Guess file type. Useful when extensions lie.
du -sh *	Size of files and folders in current directory.
df -h	Available disk space on mounted filesystems.
head file.tsv ; tail file.tsv	First or last lines.
less -S file.tsv	Scroll without wrapping long lines. Great for wide tables.
wc -l file.tsv	Count lines.
cat small.txt	Print a small file. Avoid for huge FASTQ/BAM files.
column -t -s \$'\t' table.tsv less -S	Pretty-view a TSV table.
dos2unix file.tsv	Fix Windows line endings when tools behave strangely.

Compressed / archive command	Use
zcat reads.fastq.gz head	Inspect compressed text without decompressing to disk.
zless reads.fastq.gz	Scroll through compressed text.
zgrep "TP53" annotations.gtf.gz	Search compressed text.
gzip file.fastq ; gunzip file.gz	Compress or decompress gzip files.
pigz -p 8 file.fastq	Faster parallel gzip when available.
tar -czf results.tar.gz results/ logs/ scripts/	Archive a finished run with logs and scripts.

4. Checksums and data integrity

Command / pattern	Use
md5sum file.fastq.gz	Compute MD5 checksum.
sha256sum file.fastq.gz	Compute SHA256 checksum.
md5sum -c checksums.md5	Verify files against a checksum file.
rsync -avP source/ target/	Transfer with progress and partial resume.
rsync -avP --dry-run source/ target/	Preview what would be copied before doing it.

Expert habit: after every download or copy, check file size, count files and verify checksums when available. Silent truncation is rare but devastating.

5. FASTQ, FASTA and biological sequence files

A standard FASTQ record has four lines: read identifier, sequence, plus line and quality string. Use manual inspection for learning and dedicated tools for real summaries.

Command / pattern	Use
zcat R1.fastq.gz head -n 12	Inspect the first three FASTQ records.
zcat R1.fastq.gz wc -l	Approximate read count = line count / 4 for standard FASTQ.
seqkit stats raw_data/*.fq.gz	Robust FASTQ/FASTA summary: number, length, GC and N content.
seqkit sample -n 10000 reads.fq.gz > subset.fq	Make a small teaching or debugging subset.
seqkit seq -m 100 contigs.fa > min100.fa	Keep sequences at least 100 bp.
seqkit grep -p "ACTG" sequences.fa	Find sequences by pattern or ID.
samtools faidx reference.fa	Index FASTA and create reference.fa.fai.
samtools faidx reference.fa chr1:1-1000	Extract a FASTA region.

```
# Paired-end sanity check: do all R1 files have an R2 partner?
for r1 in raw_data/*_R1*.fastq.gz; do
r2=${r1/_R1/_R2}
[ -f "$r2" ] || echo "Missing pair for $r1 -> expected $r2"
done
```

Common file types: FASTQ = reads, FASTA = sequences/reference, GTF/GFF = annotation, BED = intervals, SAM/BAM/CRAM = alignments, VCF/BCF = variants. Index files include .fai, .bai, .csi and .tbi.

6. Text tables and metadata: grep, cut, sort, uniq, awk

Command / pattern	Use
grep -i "brca1" genes.tsv	Case-insensitive search.
grep -v "control" metadata.tsv	Exclude matching lines.
grep -E "BRCA1 BRCA2" genes.tsv	Extended regular expression.
cut -f1,3 metadata.tsv	Select TSV columns 1 and 3.
cut -d, -f2 file.csv	Select a CSV column. Better: use csvkit/csvtk for messy CSV.
sort genes.txt uniq > unique.txt	Remove duplicate adjacent lines after sorting.
sort genes.txt uniq -c sort -nr	Count values and sort descending.
cut -f1 metadata.tsv sort uniq -d	Find duplicate sample IDs.
awk -F"\t" '{print \$1,\$3}' metadata.tsv	Print columns from a TSV.
awk -F"\t" 'NR==1 \$3=="case"' metadata.tsv	Keep header plus rows where column 3 is case.
sed 's/old/new/g' file.txt	Replace text in output.
tr -d "\r" < in.tsv > out.tsv	Remove hidden Windows carriage returns.

```
# Count samples per group, assuming group is column 3
cut -f3 metadata.tsv | sort | uniq -c | sort -nr
```

```
# Check sample IDs in metadata vs FASTQ filenames
cut -f1 metadata.tsv | sort > ids_from_metadata.txt
find raw_data -name '*_R1*.fastq.gz' -exec basename {} \; | sed 's/_R1.*//' |
sort > ids_from_fastq.txt
comm -3 ids_from_metadata.txt ids_from_fastq.txt
```

Metadata trap: most RNA-seq mistakes are not aligner mistakes. They are sample-name mismatches, swapped conditions, missing replicates, wrong delimiter or unrecorded batch variables.

7. Pipes, redirection and logging

Command / pattern	Use
cmd1 cmd2 cmd3	Send output of one command into the next.
command > out.txt	Save standard output, replacing old file.
command >> out.txt	Append standard output.
command 2> err.log	Save error messages.
command > out.txt 2> err.log	Save output and errors separately.
command 2>&1 tee logs/run.log	See output live and save stdout plus stderr to a log.
time command	Measure how long a command takes.

```
mkdir -p logs results/fastqc results/multiqc
fastqc raw_data/*.fastq.gz -o results/fastqc 2>&1 | tee logs/fastqc.log
multiqc results/fastqc -o results/multiqc 2>&1 | tee logs/multiqc.log
```

Use tee for teaching: learners see output live, and you still have a log for debugging after the workshop.

8. Wildcards, loops, find, xargs and parallel

Command / pattern	Use
echo raw_data/*.fastq.gz	Preview what a wildcard expands to.
for f in *.fq.gz; do echo "\$f"; done	Basic loop over files. Quote variables.
basename sample_R1.fq.gz _R1.fq.gz	Extract sample name from filename.
find . -name "*.bam"	Find files recursively.
find . -type f -size +1G	Find files larger than 1 GB.
find . -empty	Find empty files or directories.
find . -name "*.tmp" -print	Inspect files before cleanup.
find . -name "*.tmp" -delete	Delete only after inspection.
find raw_data -name "*.fq.gz" xargs -n 1 basename	Pass many filenames to another command.
find raw_data -name "*.fq.gz" -print0 xargs -0 ...	Robust version for spaces in filenames.
parallel -j 4 fastqc {} -o results/fastqc ::: raw_data/*.fq.gz	Run many independent jobs in parallel.

```
# Run FastQC per file and make one log per sample
mkdir -p results/fastqc logs
for f in raw_data/*.fastq.gz; do
sample=$(basename "$f" .fastq.gz)
fastqc "$f" -o results/fastqc 2>&1 | tee "logs/fastqc_${sample}.log"
done
```

Shell-script rule: quote variables as "\$file". Unquoted filenames with spaces or special characters can silently break commands.

9. Downloading biomedical data

Command / pattern	Use
wget -c URL	Continue an interrupted download.
wget -O name URL	Save with a chosen filename.
wget --timestamping URL	Download only if remote file is newer.
curl -L -o file URL	Download while following redirects.
curl -O -J -L URL	Use remote filename when server provides it.
rsync -avP host:/path/ local/	Robust copy from server to local.
aria2c -x 8 URL	Parallel segmented download, if available.

For public sequencing data, SRA Toolkit is useful, but also teach the idea of ENA/NCBI/GEO/ArrayExpress accession numbers and the difference between metadata, raw reads and processed data.

SRA Toolkit command	Use
vdb-config --interactive	Configure SRA Toolkit cache and download settings.
prefetch SRR123456	Download an SRA run to local cache.
fasterq-dump SRR123456 --split-files -O raw_data	Convert SRA to paired FASTQ files.
fasterq-dump SRR123456 --split-3 -O raw_data	Handle paired and unpaired reads.
df -h ; du -sh raw_data/	Check disk before and after conversion. FASTQ can be much larger than SRA.
gzip raw_data/*.fastq	Compress after conversion.

```
mkdir -p raw_data logs
prefetch SRR123456 2>&1 | tee logs/prefetch_SRR123456.log
fasterq-dump SRR123456 --split-files -O raw_data 2>&1 | tee
logs/fasterq_SRR123456.log
gzip raw_data/SRR123456_*.fastq
seqkit stats raw_data/SRR123456_*.fastq.gz
```

Workshop rule: prepare a tiny backup FASTQ dataset. Live SRA downloads fail because of network speed, cache settings, disk quotas and institutional firewalls.

10. Read QC and trimming

This is the first complete practical victory for beginners: raw reads become interpretable HTML reports.

Command / pattern	Use
fastqc sample.fastq.gz	Run QC on one FASTQ file.
fastqc -t 4 raw_data/*.fq.gz -o results/fastqc	Run QC with 4 threads on many files.
multiqc results/ -o results/multiqc	Aggregate tool reports into one HTML report.
fastp -i R1.fq.gz -I R2.fq.gz -o R1.trim.fq.gz -O R2.trim.fq.gz	Adapter/quality trimming with built-in report.
trim_galore --paired R1.fq.gz R2.fq.gz	Wrapper around Cutadapt, common in older RNA-seq teaching.
cutadapt -a ADAPTER -A ADAPTER -o R1.trim.fq.gz -p R2.trim.fq.gz R1.fq.gz R2.fq.gz	Explicit adapter trimming.

```
mkdir -p results/qc_before results/qc_after results/multiqc logs
fastqc raw_data/*.fq.gz -o results/qc_before 2>&1 | tee logs/fastqc_before.log
fastp -i raw_data/sample_R1.fq.gz -I raw_data/sample_R2.fq.gz -o
data/sample_R1.trim.fq.gz -O data/sample_R2.trim.fq.gz --html
results/sample_fastp.html --json results/sample_fastp.json 2>&1 | tee
logs/fastp_sample.log
fastqc data/*.trim.fq.gz -o results/qc_after 2>&1 | tee logs/fastqc_after.log
multiqc results/ -o results/multiqc
```

QC interpretation questions: Which samples have unusual read counts? Adapter content? Low-quality tails? Overrepresented sequences? Duplication? Does trimming improve the problem or just hide it?

11. Reference genomes, annotations and indexes

Command / pattern	Use
samtools faidx genome.fa	Index FASTA for random access.
grep -v "^#" annotation.gtf head	Inspect annotation without comment lines.
cut -f1 annotation.gtf sort uniq head	Check chromosome names in annotation.
cut -f1 genome.fa.fai head	Check chromosome names in reference.
bgzip annotation.gff3 ; tabix -p gff annotation.gff3.gz	Compress and index GFF/GTF-like genomic intervals.
bwa-mem2 index genome.fa	Index genome for short-read DNA alignment.
hisat2-build genome.fa genome_hisat2	Index genome for splice-aware RNA-seq alignment.
STAR --runMode genomeGenerate ...	Build STAR genome index.
minimap2 -d genome.mmi genome.fa	Build minimap2 index for long reads.

Biggest hidden error: reference genome and annotation must match. GRCh37 vs GRCh38, Ensembl vs UCSC names, chr1 vs 1 and different annotation releases can invalidate results.

12. Alignments: SAM, BAM and CRAM with samtools

Command / pattern	Use
samtools view sample.bam head	Peek at alignments.
samtools view -H sample.bam	Show BAM header: reference names, read groups, programs.
samtools quickcheck sample.bam	Fast check for truncated BAM/CRAM files.
samtools flagstat sample.bam	Mapping summary statistics.
samtools stats sample.bam > sample.stats	Detailed alignment statistics.
samtools idxstats sample.bam	Reads per reference sequence.
samtools sort -@ 4 in.bam -o sorted.bam	Sort alignments.
samtools index sorted.bam	Create BAM index (.bai or .csi).
samtools view sorted.bam chr1:1-100000	Extract region.
samtools view -b -q 30 -F 4 in.bam > mapped_q30.bam	Keep mapped reads with MAPQ >= 30.
samtools depth sorted.bam > depth.tsv	Per-base coverage.

```
# Short-read DNA alignment example
bwa-mem2 mem -t 8 reference.fa R1.fq.gz R2.fq.gz | samtools sort -@ 4 -o
results/sample.sorted.bam
samtools index results/sample.sorted.bam
samtools flagstat results/sample.sorted.bam > results/sample.flagstat.txt
```

```
# Long-read alignment example
minimap2 -ax map-ont reference.fa reads.fq.gz | samtools sort -o
results/ont.sorted.bam
samtools index results/ont.sorted.bam
```

BAM habit: sorted BAM + index + flagstat + log is the minimum alignment output set worth keeping.

13. Variants, intervals and genome coordinates

Command / pattern	Use
<code>bgzip variants.vcf ; tabix -p vcf variants.vcf.gz</code>	Block-compress and index VCF for fast access.
<code>bcftools view variants.vcf.gz head</code>	Inspect variants.
<code>bcftools stats variants.vcf.gz > stats.txt</code>	Variant summary statistics.
<code>bcftools query -f "%CHROM\t%POS\t%REF\t%ALT\n" file.vcf.gz</code>	Extract selected VCF fields.
<code>bcftools view -r chr1:1-100000 variants.vcf.gz</code>	Subset variants by genomic region.
<code>bcftools filter -i "QUAL>30" in.vcf.gz -Oz -o q30.vcf.gz</code>	Filter variants by expression.
<code>bedtools intersect -a A.bed -b B.bed</code>	Find overlapping genomic intervals.
<code>bedtools coverage -a targets.bed -b sample.bam</code>	Coverage over target regions.
<code>bedtools genomecov -ibam sample.bam</code>	Genome-wide coverage summary.

Coordinate trap: BED is 0-based half-open; VCF/GFF/GTF are usually 1-based. Mixing coordinate systems creates one-base errors that are hard to detect.

Useful extras	Use
<code>mosdepth sample sample.bam</code>	Fast coverage summaries, very useful for WGS/panels if installed.
<code>multiqc results/</code>	Also summarises many alignment, variant and coverage outputs.
<code>bcftools norm -f reference.fa in.vcf.gz</code>	Left-align/normalise variants against a reference.

14. RNA-seq: practical terminal workflow choices

Your transcriptome ladder uses SRA, FastQC, trimming, Trinity assembly, QUAST/BUSCO, kallisto, tximport and DESeq2. Teach that as one route, then show the modern decision tree: reference-based when a good reference exists; de novo when it does not.

Scenario	Recommended route
Reference transcriptome available	Salmon or kallisto quasi/pseudo-alignment -> tximport -> DESeq2/edgeR. Fast and beginner-friendly.
Reference genome + annotation available	STAR/HISAT2 -> sorted BAM -> featureCounts -> DESeq2/edgeR. Better for splice-aware genome alignment and QC.
No good reference	Trinity de novo assembly -> assembly QC with QUAST/BUSCO -> quantification with Salmon/kallisto -> downstream annotation.
Single-cell RNA-seq	Use workflow-specific tools such as Cell Ranger, STARsolo, kallisto bustools or alevin-fry. Do not treat it as bulk RNA-seq.

```
# Salmon transcript-level quantification
salmon index -t transcripts.fa -i references/salmon_idx
salmon quant -i references/salmon_idx -l A -1 sample_R1.fq.gz -2 sample_R2.fq.gz
-p 8 -o results/salmon/sample
```

```
# Genome-aligned RNA-seq counting
STAR --genomeDir references/star --readFilesIn R1.fq.gz R2.fq.gz
--readFilesCommand zcat --runThreadN 8 --outSAMtype BAM SortedByCoordinate
--outFileNamePrefix results/star/sample_
samtools index results/star/sample_Aligned.sortedByCoord.out.bam
featureCounts -T 8 -p -s 0 -a genes.gtf -o results/counts.txt
results/star/*Aligned.sortedByCoord.out.bam
```

RNA-seq traps: wrong strandedness in featureCounts, missing biological replicates, sample sheet order mismatch, mixed genome/annotation versions and interpreting transcript-level counts without a clear summarisation strategy.

15. De novo transcriptome and genome assembly ladders

Route	Practical sequence
Transcriptome ladder	SRA -> FastQC/MultiQC -> trim -> Trinity -> QUAST/BUSCO -> Salmon/kallisto -> tximport/DESeq2 -> gene interpretation.
Genome long-read ladder	Nanopore/PacBio -> inspect/QC -> optional filter/correct -> Flye/Canu/hifiasm -> polish -> QUAST/BUSCO -> compare to reference.
Reference route	Download reference + GTF/GFF -> index -> map reads -> sort/index BAM -> flagstat/idxstats -> count or call variants.

Command / pattern	Use
<code>Trinity --seqType fq --left L1.fq,L2.fq --right R1.fq,R2.fq --CPU 8 --max_memory 50G</code>	De novo transcriptome assembly.
<code>quast.py trinity_out_dir/Trinity.fasta</code>	Assembly contiguity summary.
<code>busco -i Trinity.fasta -m transcriptome -l lineage -o busco_tx</code>	Transcriptome completeness assessment.
<code>flye --nano-raw reads.fq.gz -o flye_asm -t 16</code>	Nanopore genome assembly.
<code>flye --pacbio-raw reads.fq.gz -o flye_asm -t 16</code>	PacBio raw-read assembly.
<code>hifiasm -o asm -t 32 reads.fq.gz</code>	PacBio HiFi assembly.
<code>medaka_consensus -i reads.fq.gz -d assembly.fa -o medaka -t 8</code>	Nanopore polishing, if appropriate.
<code>busco -i assembly.fa -m genome -l lineage -o busco_genome</code>	Genome completeness assessment.

Assembly judgement: N50 is not quality by itself. Look at total size, number of contigs, BUSCO completeness/duplication, contamination, read support and whether the result answers the biological question.

16. Long-read, metagenomic and other common bioinformatics tools

Command / pattern	Use
<code>NanoPlot --fastq reads.fq.gz -o results/nanoplot</code>	Nanopore read length and quality overview.
<code>chopper -q 10 -l 1000 < reads.fq.gz gzip > filt.fq.gz</code>	Filter ONT reads by quality and length.
<code>filtlong --min_length 1000 reads.fq.gz > filt.fq</code>	Alternative long-read filtering.
<code>kraken2 --db DB --paired R1.fq.gz R2.fq.gz --report sample.report > sample.kraken</code>	Taxonomic classification for metagenomics.
<code>bracken -d DB -i sample.report -o sample.bracken</code>	Abundance estimation after Kraken2.
<code>megahit -1 R1.fq.gz -2 R2.fq.gz -o megahit_out</code>	Metagenome assembly.
<code>qiime tools import ...</code>	QIIME 2 workflows are terminal-first but structured around artefacts.

Know when not to reinvent: mature workflows exist for RNA-seq, metagenomics, single-cell and variant calling. Your terminal skill lets you inspect, run, debug and document them.

17. Remote servers, SSH and transfers

Command / pattern	Use
ssh user@server	Connect to a remote server.
ssh -X user@server	X11 forwarding if GUI forwarding is configured.
scp file user@server:/path/	Copy one file to server.
scp user@server:/path/file .	Copy one file from server.
rsync -avP local/ user@server:/path/	Robust folder transfer with resume and progress.
rsync -avP --exclude "*.bam" local/ server:/path/	Transfer while excluding large files.
tmux new -s qc	Start persistent terminal session.
tmux attach -t qc	Reconnect later.
Ctrl+B then D	Detach from tmux without killing commands.
top ; htop ; ps aux grep fastqc	Inspect running processes.
kill PID	Stop a process by process ID.

Long jobs on remote machines: never rely on an ordinary SSH window. Use tmux/screen for interactive work or SLURM/PBS/LSF for scheduled jobs.

18. HPC and SLURM survival

Command / pattern	Use
module avail	List available software modules.
module load fastqc	Load software.
module list	Show loaded modules.
module purge	Unload all modules to avoid hidden conflicts.
srun -c4 --mem=8G command	Run a command interactively with resources.
sbatch script.sh	Submit a batch job.
squeue -u \$USER	View your queued/running jobs.
scancel JOBID	Cancel a job.
sacct -j JOBID --format=JobID,State,Elapsed,MaxRSS	Inspect completed job resource use.

```
#!/bin/bash
#SBATCH --job-name=fastqc
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --time=01:00:00
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err
```

```
set -euo pipefail
module purge
module load fastqc
mkdir -p results/fastqc
fastqc -t "$SLURM_CPUS_PER_TASK" raw_data/*.fq.gz -o results/fastqc
```

HPC judgement: requesting more CPUs only helps if the tool can use them. Check the tool option: often -t, -p, -@, --threads or --cpus.

19. Environments, containers and workflow engines

Command / pattern	Use
conda env list	List conda environments.
conda create -n rnaseq	Create environment.
conda activate rnaseq	Activate environment.
mamba install -c conda-forge -c bioconda fastqc multiqc samtools	Install bioinformatics tools faster.
conda env export > environment.yml	Save environment specification.
conda env create -f environment.yml	Recreate environment.
which fastqc ; fastqc --version	Record exact executable and version.
apptainer exec image.sif fastqc --version	Run tool inside an HPC-friendly container.
docker run --rm -v "\$PWD":/work -w /work image command	Run container locally with current folder mounted.
snakemake -j 8 --use-conda --rerun-incomplete	Run a Snakemake workflow.
nextflow run workflow.nf -profile conda	Run a Nextflow workflow.
nextflow run nf-core/rnaseq -profile docker --input samplesheet.csv --outdir results	Example style for a community workflow.

Rule of thumb: use conda/mamba for teaching and light projects, containers for portability, and Nextflow/Snakemake when many samples and steps need robust reruns.

20. Git for bioinformatics scripts

Command / pattern	Use
git init	Start tracking a project.
git status	See changed files.
git add scripts/run_qc.sh	Stage a script.
git commit -m "Add QC workflow"	Save a checkpoint.
git diff	See exact text changes.
git log --oneline	View commit history.
.gitignore	Exclude large data, logs and results from Git.

```
# Example .gitignore for bioinformatics projects
raw_data/
results/
logs/
*.bam
*.bai
*.fastq.gz
*.fq.gz
*.vcf.gz
*.tmp
```

Version scripts, not huge data. Put code, sample sheets, environment files and small configs in Git. Keep raw and large derived data outside Git.

21. Scripts and reproducibility

A script is a saved set of commands. It turns a live demo into something repeatable and reviewable.

Command / pattern	Use
#!/bin/bash	Run with Bash.
set -e	Stop when a command fails.
set -u	Stop on undefined variables.

Command / pattern	Use
set -o pipefail	Detect failure inside a pipe.
shopt -s nullglob	Empty wildcard expands to nothing instead of literal pattern.
date ; whoami ; hostname ; pwd	Record run context.
tool --version	Record tool version.
history > logs/commands_used.txt	Save interactive command history.

```
#!/bin/bash
set -euo pipefail
shopt -s nullglob
```

```
mkdir -p results/fastqc results/multiqc logs
{
date
whoami
hostname
pwd
fastqc --version
multiqc --version
} > logs/run_info.txt
```

```
fastqc raw_data/*.fastq.gz -o results/fastqc 2>&1 | tee logs/fastqc.log
multiqc results/fastqc -o results/multiqc 2>&1 | tee logs/multiqc.log
```

Expert script style: make failure visible, log everything, use explicit folders, quote variables and record versions.

22. Full mini-workflow: raw reads to QC report

```
# 1. Create project
mkdir -p sra_qc_project/{raw_data,results,logs,scripts,metadata,references}
cd sra_qc_project
```

```
# 2. Download or copy data
prefetch SRR123456 2>&1 | tee logs/prefetch.log
fasterq-dump SRR123456 --split-files -O raw_data 2>&1 | tee logs/fasterq.log
gzip raw_data/*.fastq
```

```
# 3. Inspect
ls -lh raw_data/
seqkit stats raw_data/*.fastq.gz
zcat raw_data/SRR123456_1.fastq.gz | head -n 12
```

```
# 4. Run QC and aggregate reports
mkdir -p results/fastqc results/multiqc
fastqc raw_data/*.fastq.gz -o results/fastqc 2>&1 | tee logs/fastqc.log
multiqc results/fastqc -o results/multiqc 2>&1 | tee logs/multiqc.log
```

```
# 5. Save provenance
history > logs/commands_used.txt
```

End product for students: raw_data + results/fastqc + results/multiqc + logs + scripts. They should leave with an actual reproducible folder, not just notes.

23. Troubleshooting patterns

Command / pattern	Use
command not found	Check spelling, conda activate, module load, PATH and which command.
permission denied	Check ls -l. For scripts, use chmod +x script.sh.
no such file	Check pwd, ls, exact filename, case sensitivity and Tab completion.
no space left	Check df -h and du -sh *. Clean up only after inspecting.
empty output	Check input path, grep pattern, delimiter, headers and whether the tool wrote to stderr.
very slow command	Check file size, compression, network filesystem, CPUs, memory and whether the job is actually running.
segmentation fault / killed	Often memory, corrupted input or incompatible binary/library.
different chromosome names	chr1 vs 1, MT vs chrM, genome build mismatch.
featureCounts gives low counts	Wrong annotation, wrong strandedness, wrong BAM type or unmapped/unsorted reads.
SRA conversion fails	Check cache path, disk quota, accession, network and whether prefetch completed.

```
# Debug checklist
pwd
ls -lh raw_data/
du -sh raw_data results logs
df -h
which fastqc
fastqc --version
echo raw_data/*.fastq.gz
head metadata.tsv
seqkit stats raw_data/*.fastq.gz
```

Debugging mindset: do not keep changing commands randomly. Inspect input, run a tiny subset, read the log, check versions, then rerun.

24. What a beginner should memorise first

Category	Commands
Move	pwd, ls, cd, mkdir
Inspect	head, tail, less, wc, file, du, df
Compressed files	zcat, zless, zgrep, gzip, pigz
Search tables	grep, cut, sort, uniq, awk
Connect commands	, >, >>, 2>, tee
Download and verify	wget, curl, rsync, md5sum, sha256sum
Batch	*, for loop, find, xargs, parallel
Bioinformatics core	seqkit, fastqc, multiqc, prefetch, fasterq-dump, samtools, bcftools, bedtools
Reproducibility	history, chmod, scripts, conda env export, tool --version, logs
HPC	ssh, tmux, module load, srun, sbatch, queue, scancel

Final mantra: the terminal is not about memorising commands. It is about inspecting, moving, transforming and documenting biomedical data reproducibly. For scientists, the goal is trustworthy and auditable analysis.

25. One-liners experts use constantly

Command / pattern	Use
ls -lhs head	Show largest files in the current folder.
du -sh * sort -h	Folder/file sizes sorted from small to large.
find . -maxdepth 3 -name "*.log" -print	Find log files nearby.
grep -R -i "error\ fail\ warning" logs/	Search logs for problems.
tail -f logs/run.log	Watch a live log while a job runs.
wc -l *.tsv	Check line counts across tables.
cut -f1 file1.tsv sort uniq -c sort -nr head	Most frequent values in column 1.
paste file1.txt file2.txt > combined.tsv	Combine columns side by side.
comm -12 a.sorted b.sorted	Intersection of two sorted lists.
comm -23 expected.sorted observed.sorted	Items expected but missing.
printf "%s\t%s\n" sample group > samples.tsv	Create a tiny TSV safely.
watch -n 10 "squeue -u \$USER"	Refresh job queue every 10 seconds.

26. Clinical data and responsible terminal habits

Habit / command	Why it matters
Never put identifiers in public filenames	Avoid patient names, birth dates, hospital numbers and other direct identifiers.
Do not commit data to Git	Raw data, BAMs, VCFs, FASTQs and clinical tables do not belong in public repositories.
chmod -R go-rwx sensitive_project/	Remove group/other access on a sensitive folder, if appropriate on your system.
umask 077	Make newly created files private by default in the current shell session.
grep -R "PATIENT_ID" .	Search before sharing to make sure identifiers are not in files or logs.
Use institutional storage	Do not move protected biomedical data to random cloud folders or personal devices.

Privacy rule: logs can leak sensitive paths, sample IDs and parameters. Treat logs as research records, not harmless text.

27. When shell is the wrong tool

Better tool	Use it for
Use shell	Moving files, downloading, inspecting, launching tools, logging, batching and simple text filters.
Use Python/R	Complex table cleaning, statistics, modelling, plotting, joins with messy keys and reusable analysis notebooks.
Use Nextflow/Snakemake	Many samples, many steps, conditional reruns, HPC/cloud execution and shared production workflows.
Use containers	When software conflicts, portability or exact tool versions matter more than convenience.

Professional judgement: the best bioinformatician is not the person with the longest one-liner. It is the person who chooses the simplest reliable tool and leaves an analysis that someone else can audit.

28. Analysis sanity checklist

Moment	What to verify
Before running	Confirm project folder, inputs, sample sheet, reference build, annotation version, disk space and expected outputs.
After download	Check file sizes, file count, checksums if provided and whether paired-end files match.
After QC	Open MultiQC; compare samples, not just pass/fail flags.
After trimming	Check read loss. Large read loss needs biological or technical explanation.
After alignment	Record mapping rate, duplication, reference names, insert size if relevant and odd outliers.
After counting	Check total assigned reads, strandedness and whether sample names still match metadata.
Before statistics	Look at PCA/cluster plots, library sizes, batches and outliers before trusting p-values.

29. Common tool choices by data type

Data type	Terminal toolkit
Bulk RNA-seq	FastQC/MultiQC, fastp/cutadapt, Salmon/kallisto or STAR/HISAT2 + featureCounts, DESeq2/edgeR.
WGS/WES short reads	FastQC/MultiQC, BWA-MEM2/minimap2, samtools, mosdepth, bcftools/GATK.
Long-read genome	NanoPlot, chopper/fitlong, minimap2, Flye/hifiasm/Canu, Medaka/Racon/Pilon, QUAST/BUSCO.
Metagenomics	FastQC/MultiQC, host depletion, Kraken2/Bracken or MetaPhlAn/HUMAnN, MEGAHIT/metaSPAdes.
Intervals/panels	BEDTools, mosdepth, samtools depth, bgzip/tabix, bcftools.

30. The standard reproducible deliverable

```
project/
|-- raw_data/ # untouched inputs
|-- metadata/ # sample sheet and contrasts
|-- references/ # genome, annotation, indexes, checksums
|-- scripts/ # run_*.sh, Snakefile, nextflow.config
|-- logs/ # stdout, stderr, versions, commands
|-- results/ # QC, alignments, counts, reports
-- README.md # what was run, when, with which inputs
```

What you hand over: not just results, but the path from raw data to results. A colleague should be able to inspect the commands, logs, versions and sample sheet without asking you what happened.